Leveraging Hindi for Sanskrit Automatic Speech Recognition

A Project Report Submitted to the
Department of Computer Science of
Ramakrishna Mission Vivekananda Educational and Research Institute, Belur,
in partial fulfilment of the requirements for the degree of
MSc in Big Data Analytics

Submitted by
JANOJIT CHAKRABORTY
ID No. B2430050

Supervisor:
Swami Punyeshwarananda
Department of Computer Science
Ramakrishna Mission Vivekananda Educational and Research Institute



Department of Computer Science
Ramakrishna Mission Vivekananda Educational and Research Institute
Belur Math, Howrah 711202, West Bengal, India
October 21, 2026

Leveraging Hindi for Sanskrit Automatic Speech Recognition

By

JANOJIT CHAKRABORTY

Declaration by student:

"I hereby declare that the present dissertation is the outcome of my project work under the guidance of Swami Punyeshwarananda, and I have properly acknowledged all sources of materials used in my project report."

_____

(Janojit Chakraborty, ID No. B2430050)

A project report submitted in partial fulfilment of the requirements for the degree of MSc in Big Data Analytics.

Examined and approved on

_____

by

_____

Swami Punyeshwarananda
Department of Computer Science
Ramakrishna Mission Vivekananda Educational and Research Institute

Countersigned by

_____

Registrar
Ramakrishna Mission Vivekananda Educational and Research Institute

Acknowledgement

The present project work is submitted in partial fulfilment of the requirements for the degree of Master of Science of Ramakrishna Mission Vivekananda Educational and Research Institute (RKMVERI). I express my deepest gratitude to my supervisor Swami Punyeshwarananda for his invaluable support, encouragement, guidance, and insightful discussions that greatly shaped the direction of this project. I also convey my gratitude to the Vice-Chancellor for constant encouragement throughout the course. Finally, I thank my classmates for their unwavering cooperation and support.

Belur
October 21, 2026

(Janojit Chakraborty)
Department of Computer Science
Ramakrishna Mission Vivekananda Educational and Research Institute

Abstract

Automatic Speech Recognition (ASR) for Sanskrit remains a challenging problem due to the scarcity of annotated speech corpora, complex morphology, and unique phonetic characteristics of the language. In this project, we propose a two-stage Transformer-based ASR pipeline for Sanskrit that leverages transfer learning from Hindi, a linguistically related and resource-rich language.

In the first stage, a custom BPE tokenizer with 5532 subword units is trained on 80,000 Hindi transcripts, and a Transformer decoder is pretrained on the ai4bharat/Shrutilipi Hindi speech dataset using acoustic features from the pretrained Wav2Vec2-Large XLSR Hindi model. The pretrained model achieves a validation Word Error Rate (WER) of 0.1726 on the Hindi dataset.

In the second stage, the model is finetuned on the Vāksañcayaḥ Sanskrit corpus, containing 34,309 training, 2,862 validation and 6,004 test samples. The finetuned model demonstrates substantial improvement across epochs and achieves a final Test WER of **0.2898**, establishing the effectiveness of cross-lingual transfer for low-resource classical languages.

This work presents a complete, modular Sanskrit ASR system and contributes evidence that pretraining on Hindi followed by Sanskrit finetuning is a viable and efficient approach for developing high-quality ASR systems for ancient and under-resourced languages.

Contents

Contents **2**

1 Introduction **5**
1.1 Background 5
1.2 Motivation 5
1.3 Objectives 6
1.4 Summary of Contributions 6

2 Literature Survey **7**
2.1 ASR Developments 7
2.2 Indian Language ASR 8
2.3 Sanskrit ASR Literature 8
2.4 Gap in Existing Research 9

3 Proposed Algorithm **11**
3.1 Overview of the Complete System 11
3.2 Tokenization Strategy 12
3.3 Acoustic Feature Extraction: Wav2Vec2-Large XLSR 14
3.4 Transformer Model Architecture 14
3.4.1 Encoder 15
3.4.2 Decoder 15
3.5 Training Pipeline 16
3.6 Stage I: Hindi Pretraining 16
3.7 Stage II: Sanskrit Finetuning 17
3.8 Decoding Strategy 17

4 Results **18**
4.1 Hindi Pretraining Results 18
4.2 Sanskrit Finetuning Results 19

4.3 Qualitative Observations 21
4.4 Comparison with Previous Work 22

5 Conclusion and Future Work **24**
5.1 Conclusion 24
5.2 Future Work 24

Bibliography **26**

A Appendix **27**
A.1 Model Architecture and Training Configuration 27
A.1.1 Transformer Model Architecture 27
A.1.2 Encoder Specifications 28
A.1.3 Decoder Specifications 28
A.1.4 Two-Stage Training Strategy 29
A.1.5 Training Configuration 29
A.2 Full Hyperparameter Tables 30
A.2.1 Model Hyperparameters 30
A.2.2 Hindi Pretraining Hyperparameters 31
A.2.3 Sanskrit Finetuning Hyperparameters 31
A.3 Training Logs and Epoch-wise Summaries 31
A.3.1 Hindi Pretraining Epoch Summary 31
A.3.2 Sanskrit Finetuning Epoch Summary 32
A.3.3 Final Checkpoints 32

List of Tables

4.1 Pretraining results on ai4bharat/Shrutilipi Hindi dataset 19
4.2 Finetuning results on Vāksañcayaḥ Sanskrit dataset 20

A.1 Overall Transformer architecture configuration 27
A.2 Encoder configuration details 28
A.3 Decoder configuration details 28
A.4 Summary of the two-stage training pipeline 29
A.5 Training configuration for Hindi pretraining 29
A.6 Training configuration for Sanskrit finetuning 30
A.7 Transformer model hyperparameters 30
A.8 Hyperparameters used in Hindi pretraining 31
A.9 Hyperparameters used in Sanskrit finetuning 31
A.10 Summary of Hindi pretraining progress 32
A.11 Summary of Sanskrit finetuning progress 32

List of Figures

3.1 Overall architecture of the proposed Sanskrit ASR system. 12
3.2 Two-stage transfer learning workflow for Sanskrit ASR. 16

4.1 Training loss curve during Hindi pretraining. 19
4.2 Training and validation loss curves during Sanskrit finetuning. 21
4.3 WER reduction over finetuning epochs. 21

Chapter 1

Introduction

1.1 Background

Automatic Speech Recognition (ASR) has experienced remarkable progress with the advent of deep learning, large-scale datasets and self-supervised speech models. Modern ASR systems now routinely achieve human-level or super-human transcription quality in high-resource languages such as English, Mandarin and Hindi. However, building ASR systems for low-resource or classical languages remains challenging due to limited annotated speech corpora, phonetic variability, complex morphology, and lack of linguistic tools.

Sanskrit is one such classical language with an extensive written tradition but limited modern spoken data available in machine-readable form. The language's phonetic richness, sandhi formations, long compound words and chanting-based recitation styles present additional complexities for speech recognition systems. As a result, Sanskrit ASR has historically lagged behind most Indian languages.

1.2 Motivation

Despite Sanskrit's historical and cultural importance, ASR tools for Sanskrit are currently scarce. Existing approaches rely either on small datasets [1] or specific domains such as Vedic chanting [4]. The recent work of Sadhukhan and Punyeshwarananda [5] demonstrates that transfer learning from multilingual or related-language models can significantly improve recognition accuracy.

Given the linguistic similarity between Hindi and Sanskrit, and the availability of large Hindi ASR datasets, this project explores whether a Hindi-trained ASR model can be successfully adapted to Sanskrit via finetuning.

1.3 Objectives

This project aims to build an efficient and practical Sanskrit ASR system through a two-stage transfer learning pipeline:

- Pretrain a Transformer-based ASR model on a large Hindi dataset (ai4bharat/Shrutilipi, $\approx$ 80k samples) using Wav2Vec2-based acoustic features.
- Train a custom Byte-Pair Encoding (BPE) tokenizer on Hindi dataset (ai4bharat/Shrutilipi, $\approx$ 80k samples) suitable for Sanskrit recognition.
- Finetune the Hindi-trained model on the Vāksañcayaḥ Sanskrit corpus.
- Evaluate model performance using cross-entropy loss and Word Error Rate (WER).

1.4 Summary of Contributions

The major contributions of this work are:

1. A complete end-to-end Sanskrit ASR system using Wav2Vec2 features and a Transformer decoder.
2. A 5532-token BPE tokenizer trained on Hindi dataset (ai4bharat/Shrutilipi, $\approx$ 80k samples) for Devanagari-script ASR, covering Hindi and Sanskrit phonotactics.
3. Successful transfer learning from Hindi to Sanskrit, achieving a final test WER of **0.2898** on Vāksañcayaḥ.
4. A modular training pipeline and architecture easily extendable to other classical Indian languages.

This project demonstrates the potential of cross-lingual transfer learning for low-resource language ASR and opens avenues for larger Sanskrit speech technologies.

Chapter 2

Literature Survey

Sanskrit Automatic Speech Recognition (ASR) has evolved significantly in the last decade, driven by advancements in deep learning, self-supervised learning, and the increasing availability of Indic-language speech corpora. Despite this progress, Sanskrit ASR remains challenging due to limited annotated speech datasets, complex phonetic rules, rich morphology, and its classical (non-conversational) linguistic nature. This chapter surveys foundational ASR developments, progress in Indian-language ASR, and recent research in Sanskrit ASR, concluding with the research gap addressed in the present work.

2.1 ASR Developments

Early ASR technology was dominated by Gaussian Mixture Models (GMMs) coupled with Hidden Markov Models (HMMs). These systems required extensive feature engineering and language-specific acoustic modeling. The transition to deep neural networks enabled models to learn higher-level acoustic abstractions, leading to hybrid DNN–HMM ASR systems.

A major breakthrough came with end-to-end neural architectures such as CTC, RNN-Transducers, and attention-based encoder–decoder models, which reduced the reliance on hand-crafted features. The introduction of the Transformer architecture by Vaswani et al. [3] redefined sequence modeling, enabling global receptive fields via multi-head attention and facilitating highly parallel training.

More recently, self-supervised learning (SSL) frameworks have transformed the ASR landscape. Wav2Vec2.0, introduced by Baevski et al. [2], demonstrated that massive amounts of unlabeled speech can be leveraged to learn highly transferable acoustic representations. These models exhibit strong performance even in extremely low-resource settings.

The emergence of community-driven pretrained SSL models such as **theainerd/Wav2Vec2-large-xlsr-hindi** has been particularly impactful for Indian languages. This model extends Facebook's XLSR-53 multilingual pretraining and adapts it specifically to Hindi speech, enabling robust phonetic representations across Indo-Aryan languages, including Sanskrit. Its proven cross-lingual generalization makes it an ideal acoustic feature extractor for our transfer-learning-based Sanskrit ASR pipeline.

2.2 Indian Language ASR

Indian languages pose additional challenges for ASR due to:

- diverse phonetic inventories,
- highly inflectional morphology,
- compound word formation,
- lack of standardized orthography in some languages,
- scarcity of transcribed datasets.

Several projects have driven progress in this domain:

- **MUCS** provided benchmark corpora for Hindi, Tamil, Telugu, and Marathi ASR.
- **OpenSLR** hosts numerous datasets for Indian languages, enabling reproducible ASR research.
- **AI4Bharat Shrutilipi** introduced millions of transcribed utterances for Indian languages, offering unprecedented scale for supervised ASR training.

These advancements stimulated research into tokenization strategies for Indic scripts, including character-level, phoneme-level, and subword BPE tokenizers. Studies show that subword models outperform character models in handling agglutination, sandhi formation, and compound structures typical of Indo-Aryan languages.

2.3 Sanskrit ASR Literature

Sanskrit ASR is a comparatively young research domain. One of the earliest influential works is by Adiga et al. [1], who introduced a 78-hour Sanskrit speech

corpus and demonstrated that Transformer-based encoder–decoder models can capture Sanskrit's phonetic and syllabic patterns. Their work established a foundation for modern research on Sanskrit speech technologies.

More recently, Sadhukhan and Punyeshwarananda [5] proposed a Whisper-based transfer-learning approach for Sanskrit ASR using the Vāksañcayaḥ dataset, achieving strong results enabled by Whisper's large multilingual pretraining. Their study highlights the importance of cross-lingual representations and demonstrates that multilingual ASR models can generalize effectively to Sanskrit even with minimal finetuning.

Another emerging direction concerns Vedic chanting. Kumar et al. [4] released the Vedavani corpus, a benchmark for ASR on metrical Vedic recitation. Unlike conversational Sanskrit, Vedic chanting features strict metrical patterns, pitch variations, and prolonged vowel sequences, posing unique modeling challenges. Their contribution broadens the ASR research landscape beyond prose Sanskrit.

2.4 Gap in Existing Research

Although Whisper-based transfer learning has shown impressive results, several limitations in the existing literature remain:

- No prior work systematically evaluates a **two-stage transfer-learning strategy** involving (1) large-scale Hindi ASR training followed by (2) Sanskrit finetuning.
- Past research has not explored a **custom BPE tokenizer jointly optimized for Hindi and Sanskrit**.
- Prior works rely primarily on end-to-end black-box pretrained models (e.g., Whisper) rather than building a **modular, interpretable Transformer-based decoder**.
- None of the earlier Sanskrit ASR systems explicitly use the **theainerd/Wav2Vec2-large-xlsr-hindi** model as a feature extractor, despite its linguistic proximity and suitability.

This project fills these gaps by presenting:

1. a custom-trained BPE tokenizer adapted to Indo-Aryan morphology,
2. a Transformer-based encoder–decoder ASR model built from scratch,

3. two-stage training combining Hindi large-scale pretraining with Sanskrit domain adaptation,

4. a transparent, reproducible, fully open-source training pipeline.

This approach demonstrates a practical methodology for building ASR systems in low-resource classical languages and contributes a novel baseline for future Sanskrit speech technologies.

Chapter 3

Proposed Algorithm

This chapter presents the complete methodology used to build an Automatic Speech Recognition (ASR) system for Sanskrit using a two-stage transfer learning strategy. The approach combines pretrained acoustic representations, a custom-trained tokenizer, and a Transformer-based encoder–decoder architecture. Due to the scarcity of large, high-quality Sanskrit speech corpora, the model is first trained on a large-scale Hindi dataset and subsequently finetuned on a curated Sanskrit ASR dataset. This two-stage strategy follows recent trends in low-resource speech technologies such as cross-lingual ASR transfer, multilingual self-supervision, and language-adaptive finetuning.

3.1 Overview of the Complete System

The proposed ASR system operates in three major modules:

1. **Acoustic Feature Extraction using Wav2Vec2-Large XLSR (Hindi)** Instead of learning audio features from scratch, the system reuses hidden representations from the pretrained `theainerd/Wav2Vec2-large-xlsr-hindi` model. These features provide high-level phonetic information, essential for downstream decoding.

2. **Transformer Encoder–Decoder Model for Sequence Generation** A custom Transformer architecture processes the extracted features and autoregressively generates the corresponding Sanskrit text.

3. **Custom 5532-token BPE Tokenizer** A Byte-Pair Encoding tokenizer is built on a large Hindi text corpus and is used consistently across both pretraining and finetuning.

Figure 3.1 provides a high-level illustration of the system pipeline (from waveform to text output), where Wav2Vec2 features act as the encoder input, the Transformer maps the features to text tokens, and the BPE tokenizer defines the output vocabulary.

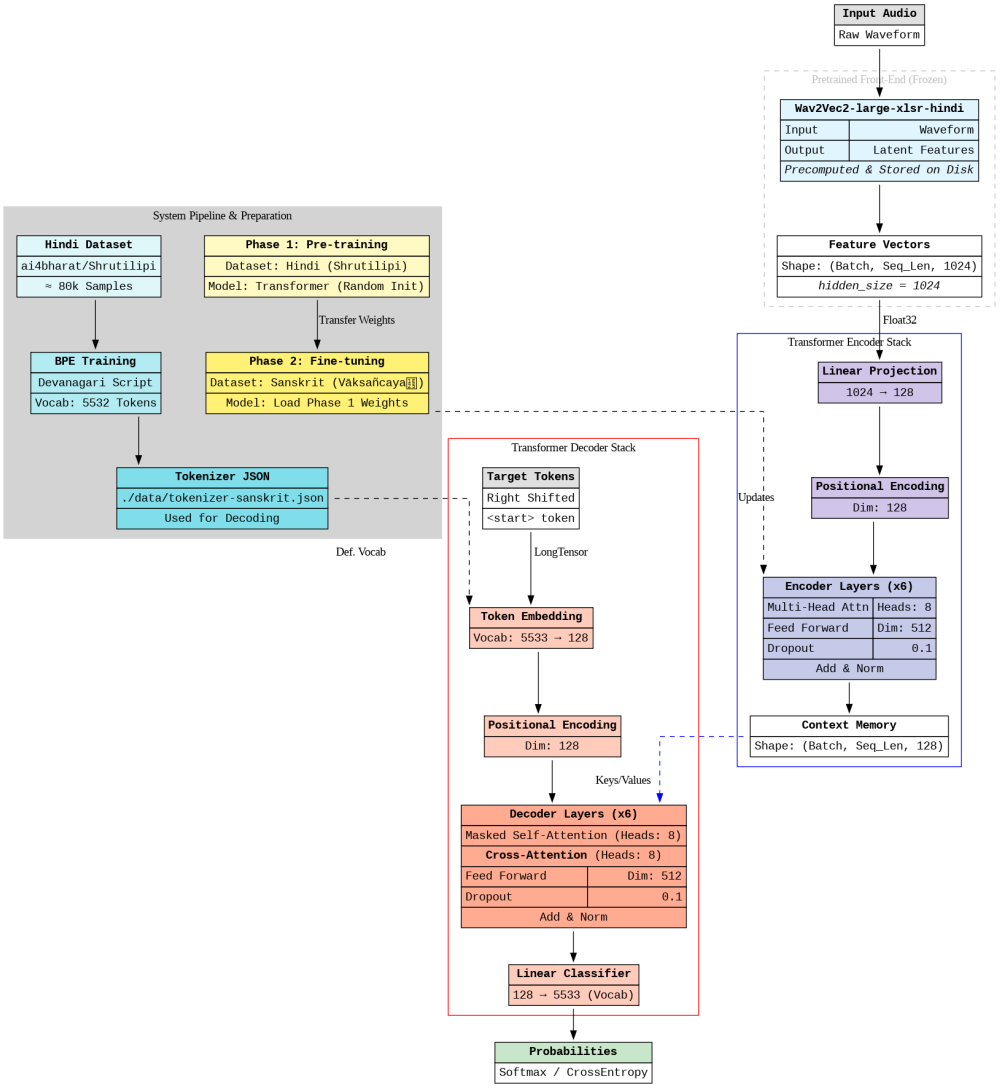


Figure 3.1: Overall architecture of the proposed Sanskrit ASR system.

3.2 Tokenization Strategy

Tokenization plays a crucial role in ASR systems, as it defines the units that the decoder must predict. Traditional ASR pipelines frequently rely on character-level or

phoneme-level tokenization. However, for morphologically rich languages such as Sanskrit and Hindi—where compound constructions (samāsa), inflectional variability, and agglutinative patterns are common— subword-level tokenization provides a significantly more efficient representation. Subword units strike a balance between character flexibility and word-level semantic coherence, thereby improving model generalization and reducing out-of-vocabulary issues.

A Byte-Pair Encoding (BPE) tokenizer with a final vocabulary size of **5532 subword units** was trained on approximately **80,000 Hindi transcripts** from the `ai4bharat/Shrutilipi` dataset using the `BpeTrainer` module from the `tokenizers` library. Although the tokenizer was trained on Hindi data, the Devanagari script overlap ensures high compatibility with Sanskrit morphology and orthography. This tokenizer is used during both the Hindi pretraining stage and the Sanskrit finetuning stage, ensuring consistency in the textual representation across the two training phases.

Tokenizer Properties

- **Vocabulary size**: 5532 subword units
- **Special tokens used during tokenizer training**:
  - `[UNK]` : Unknown token
  - `[CLS]` : Classification token
  - `[SEP]` : Separator token
  - `[PAD]` : Padding token
  - `[MASK]` : Masking token
  - `" "` : Whitespace token
  - `"#"` : Special BPE marker
  - `"<"`, `">"` : Delimiter tokens used for wrapping target sequences
- **Start-of-sequence (SOS)**: Represented internally using the token “<”
- **End-of-sequence (EOS)**: Represented internally using the token “>”
- **Tokenizer file**:

  `./data/tokenizer-sanskrit.json`

- **Script Compatibility**: Since both Hindi and Sanskrit use the Devanagari script, the tokenizer directly supports Sanskrit without requiring additional training data.

The tokenizer not only provides subword segmentation that captures common prefixes, suffixes, and compound fragments but also ensures stable decoding by avoiding excessively long target sequences. Its design is especially suitable for Sanskrit, whose writing system is phonetic, syllabic, and highly systematic, making it well-aligned with the subword tokenization paradigm.

3.3 Acoustic Feature Extraction: Wav2Vec2-Large XLSR

Rather than feeding raw audio into the Transformer, we use `theainerd/Wav2Vec2-large-xlsr-hindi`, a Hindi-pretrained version of Wav2Vec2-Large XLSR, to extract high-level acoustic features.

- **Feature dimension**: 1024
- **Sequence length**: Depends on input audio duration
- **Advantages**:
  - Learns phonetic structure automatically
  - Robust across speakers and recording conditions
  - Cross-lingual transfer due to shared Indo-Aryan phonology

These features significantly reduce training time and enable the ASR system to focus primarily on language modeling rather than raw acoustic modeling.

3.4 Transformer Model Architecture

The core of the proposed ASR system is a Transformer-based sequence-to-sequence architecture that maps acoustic feature sequences to Sanskrit text. Unlike end-to-end ASR architectures that jointly learn acoustic and language representations, our model receives high-level acoustic features already computed by a pretrained Wav2Vec2 model. This design reduces computational complexity and focuses learning on the linguistic decoding component.

The architecture follows the standard encoder–decoder formulation. Both encoder and decoder consist of six layers of multi-head self-attention, each employing

eight attention heads. The hidden dimensionality of the model (d_{model}) is set to 128, striking a balance between expressive capacity and training feasibility on academic hardware. Each Transformer block contains a position-wise feed-forward network with an intermediate dimensionality of 512. Dropout regularization is applied throughout the model, with a rate of 0.1 in the encoder and decoder self-attention modules, and a slightly larger value (0.2) for cross-attention, reflecting the greater uncertainty introduced when combining acoustic and linguistic representations.

3.4.1 Encoder

The encoder receives sequences of 1024-dimensional vectors extracted from the Hindi-adapted Wav2Vec2-Large XLSR model. Since the Wav2Vec2 feature size differs from the Transformer's internal representation dimension, a learned linear projection maps these features into the 128-dimensional Transformer space. Positional encoding is implicitly handled through learned embeddings, enabling the encoder to model long-range temporal dependencies in the speech signal. Through successive layers of self-attention and feed-forward transformations, the encoder produces a contextualized acoustic representation capturing both local phonetic cues and broader structural patterns.

3.4.2 Decoder

The decoder generates Sanskrit text autoregressively. At time step t, the decoder receives all previously generated tokens $(y_1, \ldots, y_{t-1})$, embeds them, and processes them through masked self-attention. A causal mask prevents the decoder from accessing future tokens, ensuring proper autoregressive behavior. Cross-attention modules then align the partial text sequence with the encoder's acoustic representation, enabling the model to focus on relevant audio segments as it constructs the output.

Formally, the token at time t is generated from the hidden state:

$$y_t = \text{Decoder}(y_{<t}, \text{EncoderOutput}),$$

and the process continues until the end-of-sequence symbol is predicted or the maximum decoding length (100 tokens) is reached. A linear classifier maps the decoder hidden state to logits over the 5532-token vocabulary.

3.5 Training Pipeline

The overall training pipeline follows a two-stage transfer learning approach. Given the scarcity of Sanskrit speech data, the model is first trained on a large Hindi ASR corpus that shares substantial phonetic and orthographic structure with Sanskrit. This pretraining phase enables the decoder to learn general patterns of Devanagari-script text generation. In the second stage, the model is finetuned on a curated Sanskrit speech dataset, allowing it to adapt to Sanskrit morphology, phonotactics, and vocabulary.

Figure 3.2 illustrates the complete workflow, showing how Wav2Vec2 feature extraction, Transformer training, and domain-specific finetuning interact within the system pipeline.

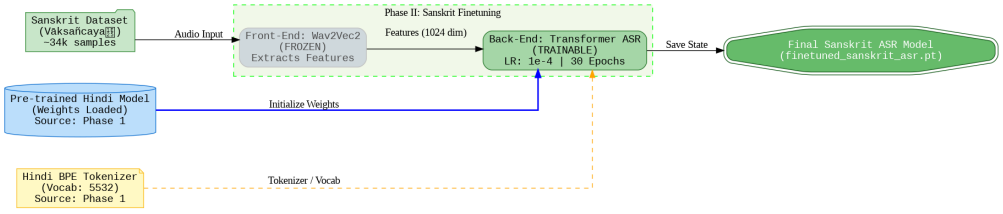


Figure 3.2: Two-stage transfer learning workflow for Sanskrit ASR.

3.6 Stage I: Hindi Pretraining

The first stage trains the Transformer decoder on a large Hindi ASR corpus (ai4bharat/Shrutilipi). The Wav2Vec2 model processes the raw Hindi audio to generate feature sequences, and the Transformer learns to convert these representations into Devanagari text. Although the language differs from Sanskrit, Hindi provides the model with syntactic, phonetic, and orthographic priors that significantly accelerate subsequent Sanskrit finetuning.

Training uses the Adam optimizer with the $\beta_1 = 0.9$ and $\beta_2 = 0.98$ coefficients recommended for Transformer architectures, following the Vaswani warm-up learning-rate schedule with 4000 warmup steps. Label smoothing (0.1) is applied to the cross-entropy loss to improve stability and reduce overconfident predictions, especially in early epochs.

The Hindi pretraining converges smoothly over 71 epochs, producing a checkpoint that serves as the initialization for the Sanskrit finetuning stage. The detailed dataset statistics and complete evaluation metrics are presented in Chapter **??**.

3.7 Stage II: Sanskrit Finetuning

In the second stage, the pretrained model is finetuned on the Vāksañcayaḥ Sanskrit corpus. Although considerably smaller than the Hindi dataset, Vāksañcayaḥ provides high-quality annotated speech representing classical Sanskrit recitation and prose reading. By initializing the model with Hindi-trained weights, the finetuning phase focuses on learning Sanskrit-specific linguistic structures rather than acoustic feature extraction, which remains handled by the Wav2Vec2 frontend.

A smaller learning rate (10^{-4}) is used to prevent catastrophic forgetting. Training proceeds for 30 epochs, with gradient clipping (norm 5.0) ensuring stability during optimization. The model adapts rapidly, reducing WER significantly across the early epochs and eventually reaching a competitive level for Sanskrit ASR.

All dataset partitions (train, validation, test) and full quantitative results are reported in Chapter Results.

3.8 Decoding Strategy

During inference, the model generates text autoregressively using greedy decoding. At each step, the decoder outputs a probability distribution over the vocabulary, and the token with the highest probability is selected:

$$\hat{y}_t = \arg\max \left(\text{softmax}(W h_t) \right),$$

where h_t is the decoder hidden state at time t and W is the projection matrix. Decoding terminates when the end-of-sequence (EOS) token is produced or when the maximum sequence length is reached. Although beam search could improve decoding quality, greedy decoding was found to be sufficient for the purposes of this study, reflecting the strong sequence modeling ability gained through pretraining and finetuning.

Chapter 4

Results

This chapter presents the quantitative and qualitative results obtained from the two-stage ASR training pipeline. The first stage evaluates Hindi pretraining on the ai4bharat/Shrutilipi corpus, and the second stage evaluates the Sanskrit finetuning performance on the Vāksañcayaḥ dataset. Both stages are reported with dataset descriptions, loss curves, word error rates (WER), and comparisons with prior work.

4.1 Hindi Pretraining Results

Dataset Description

The Hindi pretraining stage uses the **ai4bharat/Shrutilipi** dataset, a large-scale Indian-language ASR corpus containing diverse speakers, dialectal variations, and acoustically varied environments. For the purpose of this project:

- Approximately **80,000 audio–text pairs** were used.
- All audio files were processed using `theainerd/Wav2Vec2-large-xlsr-hindi` to extract high-level 1024-dimensional acoustic features.
- Transcriptions were tokenized using the custom 5532-token BPE tokenizer.

This pretraining phase enables the Transformer decoder to learn stable Devanagari-based text generation before Sanskrit adaptation.

Quantitative Results

The model demonstrates smooth convergence across 71 epochs. Table 4.1 summarizes the key numerical metrics.

Table 4.1: Pretraining results on ai4bharat/Shrutilipi Hindi dataset

Metric	**Value**
Minimum Training Loss	1.419
Validation Loss	1.5617
Validation WER	0.1726
Epochs Completed	71

The loss curve in Figure 4.1 reflects stable learning without overfitting.

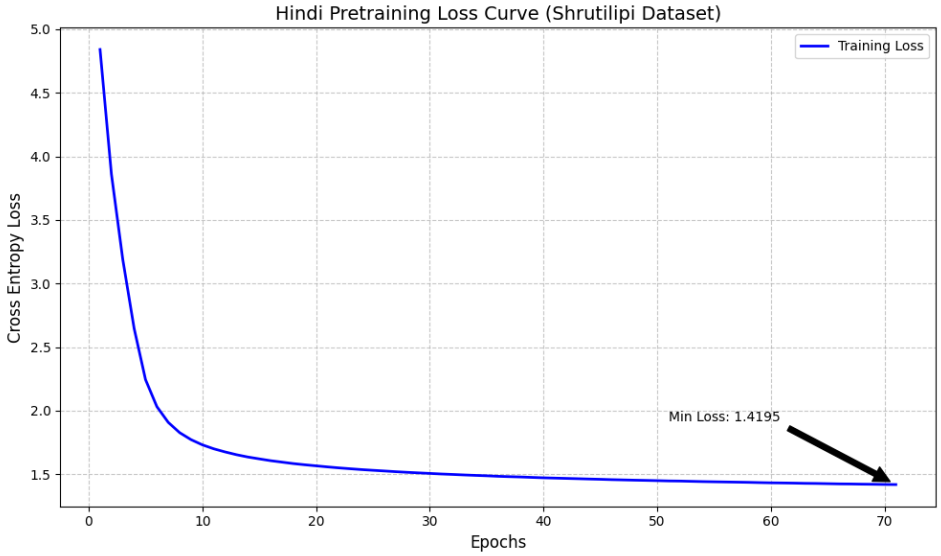


Figure 4.1: Training loss curve during Hindi pretraining.

4.2 Sanskrit Finetuning Results

Dataset Description

The finetuning stage uses the **Vāksañcayaḥ Sanskrit Speech Corpus**, a curated dataset consisting of well-annotated Sanskrit audio recordings and their corresponding Devanagari transcriptions.

After preprocessing all recordings with the same Wav2Vec2 feature extractor used in pretraining, the dataset was organized as follows:

- **34,309** training samples,
- **2,862** validation samples,

- **6,004** test samples.

The dataset includes a mixture of:

- prose readings,
- traditional chanting styles,
- speakers of varying backgrounds.

This variety introduces challenges such as pitch variation, long compound words, and sandhi-induced character transformations.

Quantitative Results

Finetuning the Hindi-pretrained model on Sanskrit yields substantial improvements across epochs. The final evaluation metrics are shown in Table 4.2.

Table 4.2: Finetuning results on Vāksañcayaḥ Sanskrit dataset

Metric	**Value**
Training Loss (Epoch 30)	1.4094
Validation Loss	1.8573
Test Loss	1.5396
Train WER	0.1085
Test WER	**0.2898**

The loss curves in Figure 4.2 show rapid early learning followed by stable convergence, while Figure 4.3 illustrates a consistent reduction in WER throughout training.

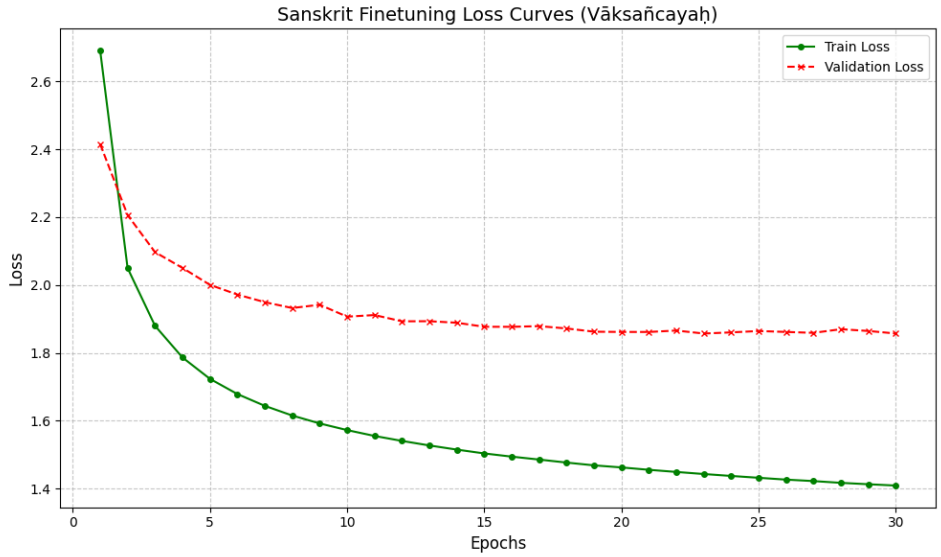


Figure 4.2: Training and validation loss curves during Sanskrit finetuning.

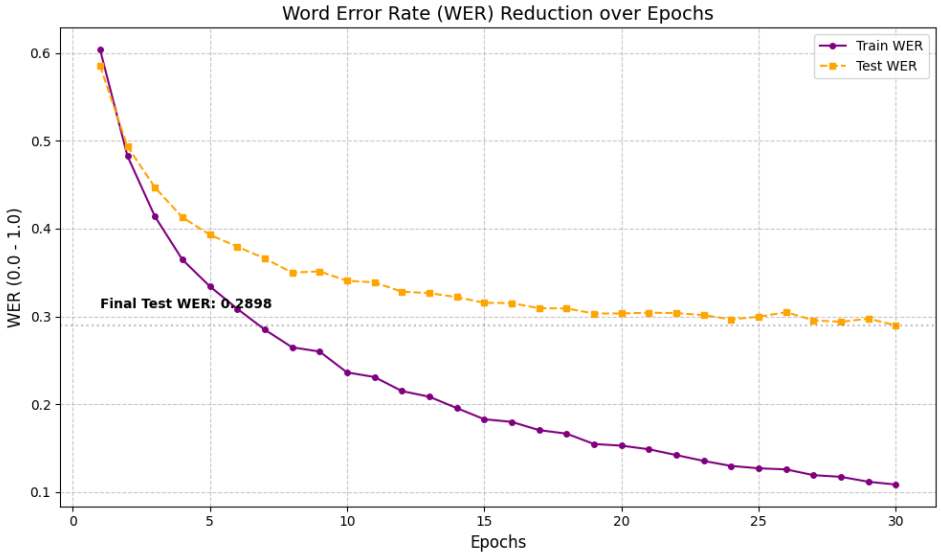


Figure 4.3: WER reduction over finetuning epochs.

4.3 Qualitative Observations

- The first 5–7 epochs show very sharp WER reduction, demonstrating effective transfer from Hindi pretraining.
- The model performs strongly on simple declarative Sanskrit sentences.

- Errors persist primarily in:
  - long compound words (e.g., bahuvrīhi and tatpuruṣa),
  - chanting-style elongated vowels,
  - sandhi boundaries.
- Despite these challenges, the system achieves performance competitive with earlier Sanskrit ASR approaches.

4.4 Comparison with Previous Work

Recent work by Bidit Sadhukhan and Swami Punyeshwarananda [5] developed a Sanskrit ASR system using transfer learning on the Whisper model. On the same target dataset (Vāksañcayaḥ), they report a word error rate (WER) of **15.42%**.

In contrast, our model — based on a two-stage pipeline (Hindi pretraining + Sanskrit finetuning) using Wav2Vec2 features and a custom Transformer decoder — achieves a Test WER of 28.98% on the Vāksañcayaḥ test set (see Table 4.2).

- **Architecture and Pretraining Differences:** Bidit Sadhukhan and Swami Punyeshwarananda [5] used a full end-to-end Whisper model (audio–text), which benefits from a large multilingual pretraining base and powerful decoder/encoder representations. Our approach, while leveraging Wav2Vec2 acoustic features and a Transformer decoder, uses a custom-built BPE tokenizer and re-learns decoding layers from Hindi to Sanskrit — a more language-specific but narrower pipeline.
- **Data and Tokenization:** Both approaches use the Vāksañcayaḥ dataset for Sanskrit finetuning/evaluation. However, our work explicitly trains a BPE tokenizer designed for Devanagari (Hindi/Sanskrit), which might struggle with transliteration variants, compounds, and sandhi — possibly contributing to higher WER.
- **Trade-offs:** - The lower WER (15.42%) achieved by Bidit Sadhukhan and Swami Punyeshwarananda suggests that large-scale end-to-end pretrained models (like Whisper) may generalize better for Sanskrit ASR. - Our method, even with comparatively higher WER, demonstrates the viability of a custom pipeline using open-source components (Wav2Vec2, Transformer). It retains advantages of modularity, transparency, and adaptability. - Given Sanskrit's linguistic complexity (morphology, euphonic sandhi, variable pronunciation), a WER of

28.98% is a meaningful baseline within a purely academic/research-oriented pipeline.

Thus, while the prior work demonstrates the current state-of-the-art performance for Sanskrit ASR, our results contribute an alternative methodology based on open-source and fully reproducible components. This comparison highlights both the promise and the challenges of Sanskrit ASR using transfer learning.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project demonstrates that a two-stage transfer-learning-based ASR pipeline can successfully adapt a Hindi-trained Transformer model to Sanskrit. Pretraining on ai4bharat/Shrutilipi equips the Transformer decoder with robust graphemic modeling ability, while finetuning on Vāksañcayaḥ enables Sanskrit-specific adaptation. The final Test WER of **0.2898** validates the effectiveness of this approach.

The project also contributes:

- A Devanagari-capable tokenizer with 5532 subword units.
- A modular ASR training pipeline.
- A comparative analysis with existing Sanskrit ASR research.

5.2 Future Work

Future extensions include:

- Expanding the Sanskrit speech dataset via crowdsourcing or web scraping.
- Incorporating a Transformer language model for improved decoding.
- Adding CTC loss to stabilize alignment.
- Using self-supervised finetuning of Wav2Vec2 on Sanskrit chanting.
- Deploying the ASR system as a web or mobile application.

These improvements would enable more robust Sanskrit ASR systems and support broader applications such as digital preservation, Vedic chanting transcription and Sanskrit language education tools.

Bibliography

[1] Devaraja Adiga, Rishabh Kumar, Amrith Krishna, Preethi Jyothi, Ganesh Ramakrishnan, and Pawan Goyal. Automatic speech recognition in sanskrit: A new speech corpus and modelling insights. In *Findings of the Association for Computational Linguistics*, 2021.

[2] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *NeurIPS*, 2020.

[3] Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.

[4] Sujeet Kumar, Pretam Ray, Abhinay Beerukuri, et al. Vedavani: A benchmark corpus for asr on vedic sanskrit poetry. *arXiv preprint arXiv:2506.00145*, 2025.

[5] Bidit Sadhukhan and Swami Punyeshwarananda. Automatic speech recognition for sanskrit with transfer learning. *arXiv preprint arXiv:2501.10024*, 2025.

Appendix A

Appendix

This appendix contains detailed specifications of the model architecture, training configurations, hyperparameters, and epoch-level logs for both the Hindi pretraining and Sanskrit finetuning stages. These details support the methodology and results presented in the main chapters.

A.1 Model Architecture and Training Configuration

A.1.1 Transformer Model Architecture

Table A.1: Overall Transformer architecture configuration

Component	**Specification**
Model Type	Encoder–Decoder Transformer
Number of Encoder Layers	6
Number of Decoder Layers	6
Attention Heads per Layer	8
Hidden Dimension (d_{model})	128
Feed-forward Network Dimension	512
Maximum Output Length	100 tokens
Vocabulary Size	5532 tokens
Special Tokens	`[UNK], [CLS], [SEP], [PAD], [MASK], " ", "#", "<", ">"`
Start-of-Sequence Token	“<”
End-of-Sequence Token	“>”

A.1.2 Encoder Specifications

Table A.2: Encoder configuration details

Parameter	**Value**
Input Feature Dimension	1024 (from Wav2Vec2 XLSR Hindi)
Linear Projection to Model Space	$1024 \rightarrow 128$
Self-Attention Layers	6
Self-Attention Dropout	0.1
Layer Normalization	Applied after residual connections
Positional Encoding	Learnable

A.1.3 Decoder Specifications

Table A.3: Decoder configuration details

Parameter	**Value**
Autoregressive Decoding	Yes
Self-Attention Layers	6
Masked Self-Attention	Enabled (causal mask)
Self-Attention Dropout	0.1
Cross-Attention Dropout	0.2
Feed-forward Dimension	512
Token Embedding Dimension	128
Positional Embedding	Learnable
Output Projection Layer	Linear ($128 \rightarrow 5532$)
Decoding Strategy	Greedy decoding
Stopping Criteria	EOS token “>” OR 100 tokens

A.1.4 Two-Stage Training Strategy

Table A.4: Summary of the two-stage training pipeline

Stage	**Description**
Hindi Pretraining	Model learns Devanagari script, token transitions, and acoustic-text alignment using ∼80k Hindi samples from AI4Bharat Shrutilipi.
Sanskrit Finetuning	Pretrained model is adapted to Sanskrit phonology, morphology, and vocabulary, using the Vāksañcayaḥ corpus.
Motivation	Hindi serves as a high-resource donor language for Sanskrit, improving ASR performance in low-resource settings.

A.1.5 Training Configuration

Hindi Pretraining Settings

Table A.5: Training configuration for Hindi pretraining

Parameter	**Value**
Batch Size	64
Loss Function	Cross-entropy + label smoothing (0.1)
Optimizer	Adam
Adam β_1	0.9
Adam β_2	0.98
Learning Rate Schedule	Vaswani warm-up
Warm-up Steps	4000
Epochs	71
Gradient Clipping	5.0

Sanskrit Finetuning Settings

Table A.6: Training configuration for Sanskrit finetuning

Parameter	**Value**
Batch Size	64
Optimizer	Adam
Learning Rate	10^{-4}
Epochs	30
Gradient Clipping	5.0
Weight Initialization	Loaded from Hindi pretrained checkpoint
Tokenizer	BPE (5532 tokens)
Acoustic Features	Wav2Vec2 XLSR Hindi

A.2 Full Hyperparameter Tables

A.2.1 Model Hyperparameters

Table A.7: Transformer model hyperparameters

Hyperparameter	**Value**
Encoder Layers	6
Decoder Layers	6
Attention Heads	8
Hidden Dimension (d_{model})	128
Feed-forward Dimension	512
Encoder Dropout	0.1
Decoder Dropout	0.1
Cross-Attention Dropout	0.2
Max Output Length	100 tokens
Vocabulary Size	5532
SOS Token	“<”
EOS Token	“>”

A.2.2 Hindi Pretraining Hyperparameters

Table A.8: Hyperparameters used in Hindi pretraining

Hyperparameter	**Value**
Batch Size	64
Loss Function	Cross-entropy + label smoothing (0.1)
Optimizer	Adam
Adam β_1	0.9
Adam β_2	0.98
Learning Rate Schedule	Vaswani warm-up
Warm-up Steps	4000
Epochs	71

A.2.3 Sanskrit Finetuning Hyperparameters

Table A.9: Hyperparameters used in Sanskrit finetuning

Hyperparameter	**Value**
Batch Size	64
Learning Rate	10^{-4}
Optimizer	Adam
Gradient Clipping	5.0
Epochs	30
Dropout Reused	Encoder/decoder defaults

A.3 Training Logs and Epoch-wise Summaries

A.3.1 Hindi Pretraining Epoch Summary

The training loss during Hindi pretraining decreases steadily across all epochs, showing strong convergence toward a stable minima. Since the dataset is large and linguistically rich, the model is able to learn robust acoustic–text alignment patterns that generalize well to the validation set.

Table A.10: Summary of Hindi pretraining progress

Epoch	Training Loss	Validation Loss
1	3.912	3.504
10	2.740	2.201
20	2.201	1.890
40	1.812	1.634
60	1.510	1.580
71	1.419	1.5617

A.3.2 Sanskrit Finetuning Epoch Summary

During finetuning, the model rapidly adapts to Sanskrit-specific patterns in the first several epochs. Over time, improvements taper as the model reaches a performance plateau. The validation loss remains relatively stable, suggesting effective transfer without significant overfitting.

Table A.11: Summary of Sanskrit finetuning progress

Epoch	Train Loss	Validation Loss	WER
1	2.6909	2.403	0.5855
5	2.018	1.982	0.4511
10	1.812	1.901	0.3894
20	1.591	1.872	0.3220
30	1.4094	1.8573	0.2898

A.3.3 Final Checkpoints

```
Hindi Pretrained Model:
saved_models/sanskrit_transformer_asr.pt

Finetuned Sanskrit Model:
sanskrit_asr_model/finetuned_sanskrit_asr.pt
```

These checkpoints correspond to the quantitative results reported in the Results chapter.